Development Scope & System Handover Summary

Project: Industrial Rotating Equipment Maintenance Intelligence Platform (v2.0.0, upgraded from v1.0.0 — not rebuilt) **Date:** 2026-11-05 · **Deployed app to update:** predictive-prescriptive-maintenance-intelligence-8vimxgmtazq78.streamlit.app

1. What was changed

1.1 Modified existing features

Area	Before (v1.0.0)	Now (v2.0.0)
Active dataset	Sidebar widget key; silently reset to NASA on page change; `dataset()` fell back to NASA test data	One central, non-widget state (`dataset_state.py`); "no dataset" is an explicit state; no fallback anywhere
Upload	Only the 14-sensor NASA schema; anything else = dead end; extra columns dropped silently	Any valid sensor data accepted; AI path only when the schema and value range fit; otherwise *sensor-only* with the reasons listed; unused columns reported
Clear session	Removed 6 keys; upload/source survived	Wipes every key, re-keys the file uploaders, clears caches; stored DB records are kept
Sensor & Degradation	EDA/reference band always from NASA data	Everything computed from the active dataset; unavailable analyses are explained
Recommendations	Technical wording, one finding type	Plain language with finding type, evidence, action, verification, status, "confirmed fault?" statement
Reports	Input mode always "Compatible engine trajectory"; included RMSE/model context	New asset/fleet PDFs stamped with source, dataset, input mode, dataset ID; no model-development metrics; stale PDFs discarded on dataset change
UI	Model-development oriented (11 pages)	Maintenance-task navigation; ML internals hidden (dev mode only)

1.2 New features

Global dataset management; sensor-only analysis (statistical drift per asset, text asset IDs, timestamps, XLSX); equipment registry (manual + CSV/Excel import with row-level rejection); maintenance management (create/edit/assign/progress/complete, audit trail, per-asset history); recommendation → maintenance record conversion with equipment↔dataset-asset linking; saved analyses and attached reports per equipment; dashboard panel with counts from stored records only.

1.3 Bugs fixed (found by inspection and by testing)

1. Data source reset after navigation (root cause: widget-key state dropped on page change).
2. Hidden NASA fallback when an upload was selected but absent.
3. Cache/status collisions across datasets (cache keyed by name + engine id; statuses by rule+engine id).
4. PDF "Input mode" label hard-coded.
5. Clear session incomplete.

6. Timestamp columns were read as numbers (nanoseconds) — caught by a unit test; fixed.
7. Duplicate timestamps silently accepted once converted to a sequence — caught by a unit test; fixed.
8. Delete-confirmation checkbox kept its state across equipment records (a second record could be deleted with one click) — caught by the browser workflow; the key is now per asset.
9. Over-long axis title overlapping the chart title on the degradation chart.

1.4 Redesigned pages

Control Room, Fleet Overview, Sensor & Degradation Analysis, Maintenance Recommendations, Reports & Export, Data Source (new), Settings (slimmed), Equipment Management (new), Maintenance Management (new). Engine Intelligence, Explainable AI, What-if and Model Performance moved to `pages/dev/` (visible only with `MAINT_DEV_MODE=1`).

---

2. Current capabilities

Existing AI (unchanged models, no retraining): XGBoost remaining-life estimate, unsupervised anomaly detector, 8 recommendation rules — for NASA C-MAPSS-compatible aircraft turbofan data only. **Sensor analysis (new, non-AI):** per-sensor drift against each asset's own early readings, in units of typical early noise; peer comparison; correlation; statistics. Status per asset: Stable / Drift detected / Insufficient history. **Equipment management:** registry with 7 equipment types, unique Asset IDs, search/filters, details (identification, specs, location, status, linked datasets, analyses, reports, maintenance), edit, protected delete, CSV/Excel import. **Maintenance management:** records with type, issue, description, action, technician, priority, planned/created/completion dates, status (Pending / In Progress / Completed / Cancelled), work performed, notes, audit trail; completed records retained. **Reporting/export:** asset PDF, fleet PDF, CSV (assets, recommendations, equipment, maintenance, rejected import rows); reports attachable to equipment records.

---

3. How the system works (user workflow)

1. Open the app: no dataset, nothing auto-loaded (landing page / "No dataset loaded" panels).
2. **Data Source** → load the built-in NASA demo, or upload CSV/TSV/TXT/XLSX → map columns → validation result (assets, records, analysis level, warnings, unused columns) → confirm → *Make this the active dataset*. The sidebar shows the active dataset on every page (Built-in NASA demo vs Uploaded data).
3. **Control Room / Fleet Overview** show KPIs, condition and priority distributions, filters, trends. **Sensor & Degradation** shows dataset overview, per-asset degradation, unusual behaviour/drift.
4. **Maintenance Recommendations** lists findings (detected condition, why, sensor evidence, priority, action, verification, "confirmed fault?", status). Status can be edited; a finding can be turned into a maintenance record for registered equipment.
5. **Equipment Management** registers/imports equipment, links it to a dataset asset, saves analyses and reports. **Maintenance Management** runs the work lifecycle.
6. **Reports & Export** creates PDFs for the active dataset only. **Clear session** returns to the clean state; stored equipment/maintenance data remain.

---

4. Technical architecture

4.1 Folder structure

```
app.py                        entry point: set_page_config, navigation (dev pages only if
MAINT_DEV_MODE=1), sidebar
pages/                        home, fleet_overview, equipment, maintenance, sensor_analysis,
recommendations, reports, data_source, settings
pages/dev/                    engine_intelligence, explainable_ai, what_if_analysis,
model_performance
src/dataset_state.py          central dataset record/token/invalidation/clear (pure, unit-
tested)
src/app_state.py              Streamlit glue: caches, activate_*, require_dataset, sidebar
src/upload_flow.py            upload assessment (validation + AI/sensor-only classification)
src/analysis_service.py       fleet view, findings, KPIs, asset detail for both capability
levels
src/generic_analysis.py       non-AI sensor drift analysis
src/storage.py                SQLite layer (equipment, maintenance, links, analyses,
reports)
src/dashboard_panels.py       stored-records dashboard panel
src/recommendation_engine.py rules + plain-language fields + finding classes
src/report_generator.py       PDFs (asset_report_pdf, summary_report_pdf; legacy
engine/fleet kept)
src/ (existing)               data_loader, data_validation, rul_target, feature_engineering,
train_models, evaluate_models, inference, explainability, anomaly_detection,
what_if_engine, fleet_analysis, engine_analysis, upload_mapping, ui_components, config
models/ artifacts/ data/ docs/ tests/ (test_pipeline, test_p1, test_p2, e2e/)
```

4.2 Data and session management

`dataset_state.py` holds the active dataset in `st.session_state["ds_active"]` (a plain key, immune to widget-state cleanup). A record contains the frame, kind (built-in / upload), capability (`ai` / `sensor_only`), sensors, labels, asset names and a **content-based token**. `activate()` deletes all dependent keys (results, selections, widget keys with dataset-dependent prefixes, action statuses); caches (`_view_cached`) and generated reports are keyed by the token, so they cannot be served for another dataset. `clear_session()` deletes every key and bumps a nonce that re-keys the uploaders. Pages call `require_dataset()`, which shows the "choose a data source" panel and stops — there is no fallback.

4.3 Database (SQLite, `storage/maintenance.db` or `$MAINT_DB_PATH`)

```
equipment(asset_id PK, name, type∈7 types, manufacturer, model, serial_number,
installation_date, location, department, status∈5, notes, created_at, updated_at)
maintenance_records(id PK, asset_id FK→equipment ON DELETE RESTRICT, type, reported_issue,
description, recommended_action, technician, priority∈{Low..Critical},
                    created_date, planned_date, status∈{Pending,In
Progress,Completed,Cancelled}, completion_date, work_performed, notes, source, source_ref,
dataset_ref, ...)
maintenance_events(id PK, record_id FK→maintenance_records, at, event, detail)          --
audit trail
asset_datasets(id PK, asset_id FK, dataset_label, source_file, dataset_token, capability,
dataset_asset, UNIQUE(asset_id, token, asset))
analysis_records(id PK, asset_id FK, dataset_*, state, rul, priority, n_findings,
last_cycle, summary)
```

```
report_records(id PK, asset_id FK, report_type, filename, dataset_*, generated_at, content BLOB)
```

`PRAGMA foreign_keys=ON`; CHECK constraints on enumerations; schema created with `IF NOT EXISTS` and `user_version` (never dropped/truncated). Asset IDs are unique case-insensitively; imports are atomic; equipment with maintenance history cannot be deleted.

4.4 How pages share data

All dataset pages call `S.view(record)` → `analysis_service.fleet_view` (cached by token + thresholds), so Control Room, Fleet Overview, Recommendations and Reports show identical numbers. Equipment/Maintenance pages talk only to `storage.py`; the link to dataset analysis is the explicit `asset_datasets` table and the recommendation → record conversion.

4.5 Dependencies

`requirements.txt`: streamlit, plotly, pandas, numpy, scipy, scikit-learn, xgboost, shap, joblib, matplotlib, reportlab, pillow, pytest, **openpyxl** (Excel), **pypdf** (tests). SQLite is part of the Python standard library.

---

5. AI model scope and limitations

Equipment type	Registered	Sensor data	Statistical sensor analysis	AI remaining-life / anomaly model
Aircraft Engines (C-MAPSS-compatible data)	Yes	Yes	Yes	**Yes** (only supported case)
Electric Motors, Pumps, Compressors, Fans, Bearings, Other Rotating Equipment	Yes	Yes (any valid sensor file)	Yes	**No — no validated model exists**

The UI shows a three-level support text per equipment record (registered only / sensor data linked / AI-supported); a pump linked to NASA data is **not** reported as AI-supported. Models were **not retrained or replaced**. Limitations: simulated single-condition training data; estimates have an upper limit and typical error; sensor drift and anomaly flags are observations, not diagnoses; no component identification; no live telemetry; recommendation action statuses are session-only (maintenance records are persistent).

---

6. Testing results

Automated unit/integration tests: `pytest -q` → **76 passed** (32 original + 20 Priority 1 + 24 Priority 2). **Browser tests (real headless Chromium against the running app):** Priority 1 journey **35/35**; Priority 2 workflow **24/24 + 8/8 after a real server restart**.

Priority 1 acceptance (13-step journey) — all passed

#	Step	Evidence
1	Open app	no dataset, no auto-load; every page shows the data-source prompt, no NASA results

#	Step	Evidence
2	Load built-in NASA	100 assets; sidebar "BUILT-IN NASA DEMO"
3	Analyse	plain-language recommendations with finding types; NASA stays active across all 7 pages
4	Upload another dataset (pumps.csv: text IDs, timestamps, 2 sensors)	accepted as sensor-only; sidebar "UPLOADED DATA · pumps.csv"
5	Navigate every page	all 7 pages show pumps.csv, no exceptions, no NASA text
6–7	Recommendations / export PDF	drift finding for PUMP-A only; PDF generated
8	PDF matches active dataset	contains "UPLOADED DATA", file name, PUMP-A; no "RMSE", "SHAP", NASA or old mode label
9–10	Clear session	sidebar reset; all pages empty; no stale report/download; uploader empty
11–13	New upload (NASA-compatible) and repeat	AI-supported; correct input-mode in PDF; old PDF not offered after a dataset change; replacing NASA by an upload leaves no NASA results
Invalid inputs: unsupported type (.pdf), empty file, 1-column file, file without measurement columns, missing value → clear message, active dataset unchanged, no activate path. Unit tests add: missing sensor and wrong-unit NASA files → classified sensor-only with explicit reasons, duplicates, non-numeric values, bad timestamps.		

Priority 2 acceptance (15-step workflow) — all passed

Register manually ✔ · duplicate ID rejected ✔ · CSV import (2 valid, 3 rejected with reasons, imported valid only) ✔ · Excel import ✔ · malformed file handled ✔ · search and type filter ✔ · details ✔ · edit ✔ · create maintenance record ✔ · assign technician ✔ · Pending→In Progress ✔ · work notes ✔ · Completed with completion date ✔ · per-asset history reached from the equipment record ✔ · delete blocked for equipment with history, allowed with confirmation otherwise ✔ · **app restart: registry, record, technician, notes and dashboard counts persisted** ✔ · dashboard counts (3 registered, 1 completed, 0 pending, 0 critical) ✔ · invalid Asset ID characters rejected ✔ · recommendation → maintenance record created and linked ✔. Invalid references (unknown Asset ID, foreign-key violation at DB level) are covered by unit tests.

Known issues / incomplete: (a) Streamlit Cloud storage is ephemeral — see §7; (b) recommendation action statuses are session-only; (c) no authentication/roles; (d) the browser tests were run locally, the deployed Cloud instance still has to be updated by you (push + redeploy) and smoke-tested there; (e) the Cloud deployment shows the app navigation with 9 pages; dev pages hidden unless the env var is set; (f) imports reject (do not update) already-existing Asset IDs.

---

7. Running and deployment

Local: `pip install -r requirements.txt` → `pytest -q` → `streamlit run app.py` (http://localhost:8501). Env: `MAINT_DEV_MODE=1` shows internal pages; `MAINT_DB_PATH` sets the database file. **Streamlit Community Cloud:** push the project to GitHub, app entry `app.py`, Python 3.11+, requirements from `requirements.txt`; no secrets needed. To update the existing app, commit and push to its tracked branch — it redeploys automatically. **Persistence caveat:** the Cloud container file system is temporary; the SQLite file persists across app restarts only while the container lives and is lost on redeploy/rebuild. For durable data use a persistent database (next step: a hosted PostgreSQL via the same `storage.py` interface) or export CSVs regularly.

---

8. Final project scope

Delivered: Priority 1 (stabilisation) and Priority 2 (equipment + maintenance management) as specified, tested, with the existing NASA C-MAPSS module preserved. **Deliberately not done (per your rules):** no new AI models or retraining; no AI for other equipment types; no fake data; no live telemetry; no silent NASA fallback; no destructive DB changes. **Recommended next steps:** persistent hosted database for Cloud; user roles; persisting recommendation statuses through maintenance records only; validated models per equipment type when real labelled data exists.